

Party Time

- Projekt
- Abnahme der Anforderungen
- Meilensteine
- User Story im Detail
- Pipeline
- Pull Request
- Vorstellung Backend
- Vorstellung Frontend

Anforderungen als Projekt in GitHub

	Title	Status	Linked pull requests	Labels	Priority	Stakeholder
1	✓ F001 Events anlegen #1	✓ Done	🔗 #61	enhancement	Hoch	Veranstalter
2	✓ F002 Events bearbeiten #2	✓ Done	🔗 #64	enhancement	Hoch	Veranstalter
3	✓ F003 Events löschen #3	✓ Done	🔗 #65	enhancement	Hoch	Veranstalter
4	✓ F004 Gäste einladen #4	✓ Done	🔗 #67 🔗 #71	enhancement	Hoch	Veranstalter, Gäste
5	✓ F005 Gäste ausladen #5	✓ Done	🔗 #69 🔗 #71	enhancement	Hoch	Veranstalter
6	✓ F006 Teilnehmer überblicken #6	✓ Done	🔗 #68 🔗 #71	enhancement	Hoch	Veranstalter
7	✓ F007 Einladung erhalten #7	✓ Done			Hoch	Nutzer der Plattform
8	✓ F008 Zusage zum Event geben #8	✓ Done			Hoch	eingeladener Nutzer
9	✓ F009 Absage zum Event geben #9	✓ Done			Hoch	eingeladener Nutzer
10	✓ F010 Konto erstellen #10	✓ Done	🔗 #32		Hoch	neue Nutzer
11	✓ F011 Konto anmelden #11	✓ Done	🔗 #49	enhancement	Hoch	Nutzer der Plattform
12	✓ F012 Konto abmelden #12	✓ Done	🔗 #55		Hoch	Nutzer der Plattform
13	✓ F013 Passwort ändern #13	✓ Done	🔗 #54		Niedrig	Nutzer der Plattform
14	✓ F014 Konto verifizieren #14	✓ Done	🔗 #43		Optional	Nutzer u. Betreiber der P...
15	✓ F015 Konto löschen #15	✓ Done	🔗 #53		Optional	Nutzer der Plattform
16	✓ F016 Events überblicken #16	✓ Done	🔗 #66 🔗 #71	enhancement	Niedrig	Teilnehmer eines Events



Anforderung lt. Pflichtenheft

Abnahme einer implementierten Anforderung

F002 Events bearbeiten #2

Buttons: Closed, 1 of 1, #64, party-time-2/party-time, Public

Header: Edit, Copy, Star, More, Close

Activity Log:

- JulianTurner opened on Mar 30, 2023
- JulianTurner edited on Mar 30, 2023
- JulianTurner added this to Party-Time on Mar 30, 2023
- JulianTurner added this to the Event milestone on Mar 30, 2023
- JulianTurner assigned E3FxGaming, dangllucas and JulianTurner on Apr 17, 2023
- JulianTurner added enhancement on May 20, 2023
- dangllucas moved this to In progress in Party-Time on May 26, 2023
- dangllucas linked a pull request that will close this issue F002 events bearbeiten #64 on May 26, 2023
- JulianTurner closed this as completed in #64 on Jun 14, 2023
- github-project-automation moved this from In progress to Done in Party-Time on Jun 14, 2023
- JulianTurner mentioned this on Jul 17, 2023
- F017 Benachrichtigungen zum Event #17

Description:

Die Plattform soll Veranstaltern die Möglichkeit geben, bereits erstellte Events zu bearbeiten und zu aktualisieren, indem sie die ursprünglich eingegebenen Informationen ändern oder ergänzen.

Diese Funktion ermöglicht es Veranstaltern, ihre Events flexibel an Veränderungen oder Fehler anzupassen, ohne dass sie ein neues Event erstellen müssen.

☒ Veranstalter können ein bereits erstelltes Event auswählen und die entsprechenden Informationen bearbeiten und speichern.

Assignees: E3FxGaming, JulianTurner, dangllucas

Labels: enhancement

Projects: Party-Time (Status: Done), Iteration: Choose an iteration, Estimate: Enter number..., Priority: Hoch, Stakeholder: Veranstalter

Milestone: Eventfunktionalität (Veranstalter) (Due on Jun 15, 100% complete)

Development: F002 events bearbeiten (party-time-2/party-time)

Notifications: Unsubscribe

You're receiving notifications because you're

User Story

Erforderlich für die Abnahme

Abnahmekriterien

Meilenstein Referenz

Deployment

Gliederung des Projekts in Meilensteine

- Funktionalität baut aufeinander auf
 - Basisfunktionalität der Plattform
 - Eventfunktionalität (Veranstalter)
 - Eventfunktionalität (eingeladener Nutzer)
 - Eventfunktionalität (Teilnehmer eines Events)

User Story im Detail

Die Plattform soll es Benutzern ermöglichen, Events zu erstellen, indem sie folgende Informationen angeben: Name des Events, Datum, Uhrzeit, Veranstaltungsort.

Die Möglichkeit, Events auf der Plattform zu erstellen, ist eine Kernfunktion, die es Veranstaltern ermöglicht Events zu planen und zu organisieren. Diese Funktion kann zu einer steigenden Nutzerzahl beitragen.

- ✓ Veranstalter sollten in der Lage sein, ein neues Event anzulegen, indem sie ein Formular ausfüllen, in dem sie alle relevanten Informationen zum *Event* (Name des Events, Datum, Uhrzeit, *Veranstaltungsort*) angeben.

- | | | |
|--|---------------------------------------|--|
| 1. Welches Ziel soll zum Erfüllen der Anforderungen erreicht werden? | 2. Warum ist die User Story relevant? | 3. Welche Abnahme-Kriterien muss das Programm erfüllen, um die Durchführung der User Story zu ermöglichen? |
|--|---------------------------------------|--|

Pipeline beim Check zur Qualitätssicherung

- Bestehen ist notwendig für den Pull Request
- Prüft auf:
 - Format des Codes (editor.config)
 - Linting Errors JS (ES-Lint)
 - Tests (Unit-Tests: JUnit, Jest)
 - End-to-End (Cypress)

Nx Cloud Report

CI is running/has finished running commands for commit [a90a23b](#). As they complete they will appear below. Click to see the status, the terminal output, and the build insights.

 [See all runs for this branch](#)

▼ Successfully ran 5 targets

- [nx affected --target=e2e --parallel=4 --ci](#)
- [nx affected --target=build --parallel=4](#)
- [nx affected --target=test --parallel=4 --ci --code-coverage](#)
- [nx affected --target=lint --parallel=4](#)
- [nx-cloud record -- npx nx format:check](#)

Review des Pull-Request

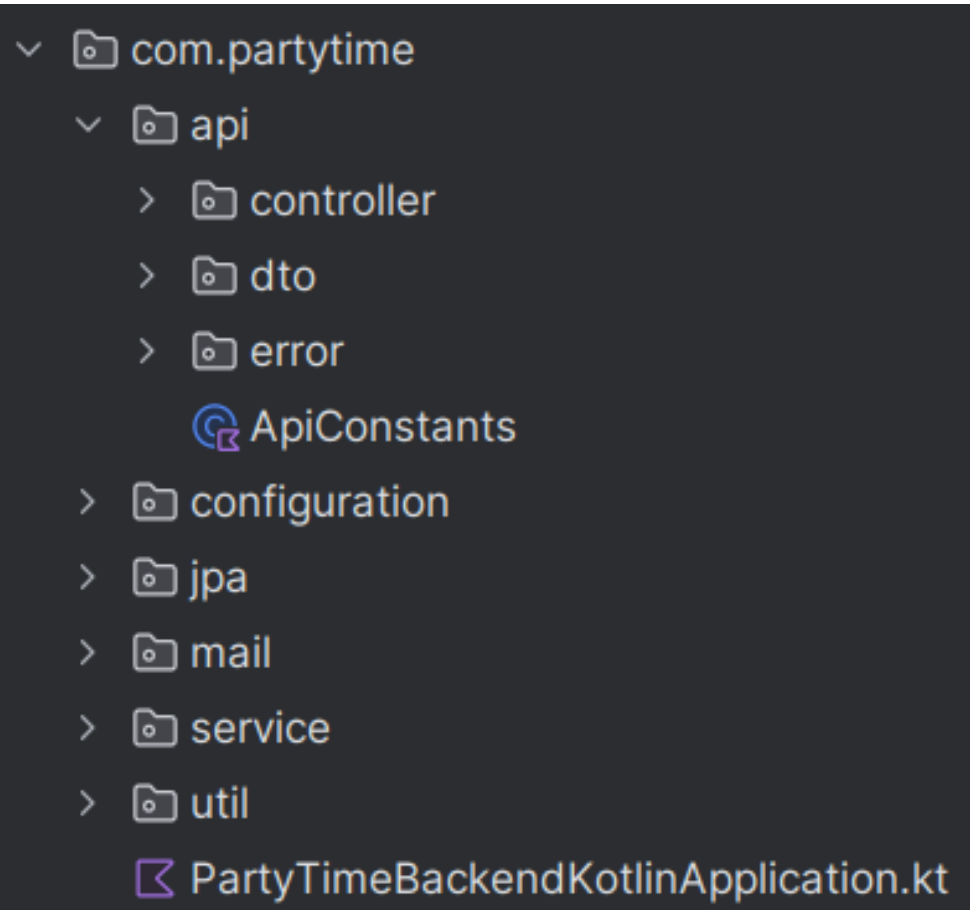
- Merge des Pull Request geht nur mit dem 4 Augen Prinzip (separation of duties)

	Review required At least 1 approving review is required by reviewers with write access. Learn more about pull request reviews.	Add your review
	No unresolved conversations There aren't yet any conversations on this pull request.	View
	All checks have passed 13 successful checks	Show all checks
	Merging is blocked Merging can be performed automatically with 1 approving review.	

Backend

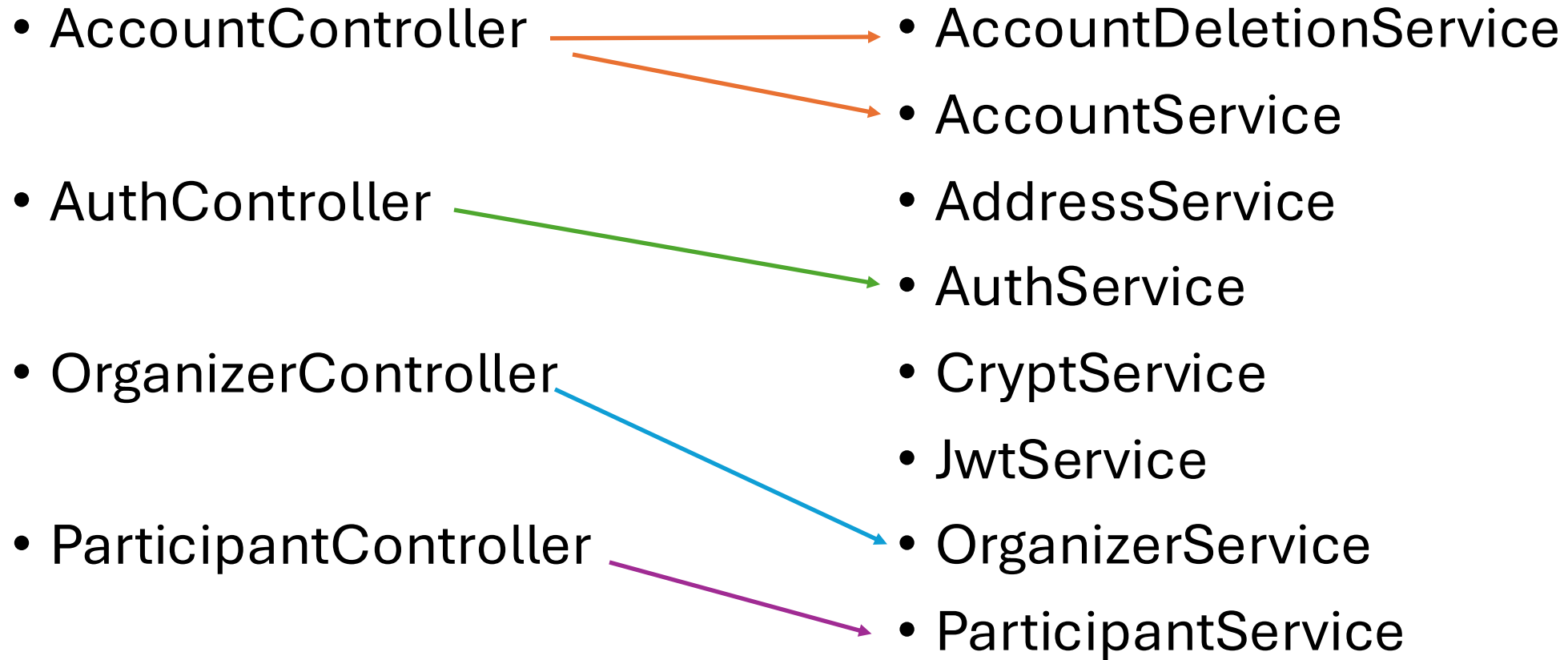
- Projektstruktur
- Controller & Services
- Validierung
- Tests
- Coverage

Backend - Projektstruktur



- Spring Anwendung mit MVC-Pattern
 - "api" -> Frontend-relevante Inhalte
 - "configuration" -> z. B. Konfiguration für SpringSecurity
 - "jpa" -> Datenbank-Peristenz
 - "mail" -> zentralisierter Mail-Versand mit Mustache-Templates
 - "service" -> Business-Logik/View-Layer
 - "util" -> Helfer/Extension-Functions

Backend – Controller & (autowired) Services



Backend - Validierung

- Validierung auf Controller-Ebene & Datenbank-Ebene
- Jakarta Bibliothek für die Beschreibung von Regeln
- Einfache Regeln wie Min-/Max-Text-Länge, aber auch komplexe Regeln wie z. B. das ein String einer E-Mail-Adresse entspricht

```
data class AccountRegisterDTO(  
    @field:NotEmpty  
    @field:Size(min = 5, max = 20)  
    val name: String,  
  
    @field:NotEmpty  
    @field:Email  
    val email: String,  
  
    @field:NotEmpty  
    @field:Size(min = 8, max = 30)  
    @field:Pattern(regex = ApiConstants.REGEX_PASSWORD)  
    val password: String  
)
```

Backend – zu testender Code

```
/**
 * Checks if extracted claims are valid.
 *
 * @param claims Instance of [Claims] whose validity should be checked
 * @return Information about the validity of the claims
 */
fun isValid(claims: Claims): Boolean = claims.run {
    issuer.equals(ISSUER)
        && id != null
        && issuedAt.before(Date())
        && subject != null
        && get(CLAIM_EMAIL) != null
}
```

Backend – Test Erfolgs-Fall

```
@Test
fun isValidTrue() {
    //setup - mock
    every { claims.issuer } returns JwtService.ISSUER
    every { claims.id } returns "Not null"
    val dateThreeMonthsAgo = Calendar.getInstance().apply {
        time = Date()
        add(Calendar.MONTH, amount: -3)
    }.time
    every { claims.issuedAt } returns dateThreeMonthsAgo
    every { claims.subject } returns "Not null"
    every { claims[JwtService.CLAIM_EMAIL] } returns "Not null"
    //execute
    assertTrue(jwtService.isValid(claims))

    //verify
    verify(exactly = 1) { claims.issuer }
    verify(exactly = 1) { claims.id }
    verify(exactly = 1) { claims.issuedAt }
    verify(exactly = 1) { claims.subject }
    verify(exactly = 1) { claims[JwtService.CLAIM_EMAIL] }
}
```

Backend – Test ArgumentProvider

```
@JvmStatic
fun isValidFalseArgumentProvider(): Stream<Arguments> {
    val dateThreeMonthsAgo = Calendar.getInstance().apply {
        time = Date()
        add(Calendar.MONTH, amount: -3)
    }.time

    val dateThreeMonthsFuture = Calendar.getInstance().apply {
        time = Date()
        add(Calendar.MONTH, amount: 3)
    }.time

    return Stream.of(
        Arguments.of(...arguments: "Not Issuer", "Not null", dateThreeMonthsAgo, "Not null", "Not null"),
        Arguments.of(JwtService.ISSUER, null, dateThreeMonthsAgo, "Not null", "Not null"),
        Arguments.of(JwtService.ISSUER, "Not null", dateThreeMonthsFuture, "Not null", "Not null"),
        Arguments.of(JwtService.ISSUER, "Not null", dateThreeMonthsAgo, null, "Not null"),
        Arguments.of(JwtService.ISSUER, "Not null", dateThreeMonthsAgo, "Not null", null),
    )
}
```

Backend – Test Fehler-Fall

```
@ParameterizedTest
@MethodSource("com.partytime.service.JwtServiceUnitTest#isValidFalseArgumentProvider")
fun isValidFalse(issuer: String, id: String?, issuedAt: Date, subject: String?, email: String?) {
    //setup - mock
    every { claims.issuer } returns issuer
    every { claims.id } returns id
    every { claims.issuedAt } returns issuedAt
    every { claims.subject } returns subject
    every { claims[JwtService.CLAIM_EMAIL] } returns email

    //execute
    assertFalse(jwtService.isValid(claims))

    //verify
    verify(atLeast = 0, atMost = 1) { claims.issuer }
    verify(atLeast = 0, atMost = 1) { claims.id }
    verify(atLeast = 0, atMost = 1) { claims.issuedAt }
    verify(atLeast = 0, atMost = 1) { claims.subject }
    verify(atLeast = 0, atMost = 1) { claims[JwtService.CLAIM_EMAIL] }
}
```

Backend - Coverage

- Prüfung der Backend Code Coverage mit der Bibliothek "Kover"
- Bereits 100 % Coverage der Services
- Wird noch stark wachsen wenn mit Integration-Tests die gesamte Anwendung getestet wird

party-time-backend: Overall Coverage Summary

Package	Class, %	Method, %	Branch, %	Line, %	Instruction, %
all classes	80.3% (61/76)	70.8% (136/192)	47.5% (58/122)	71.8% (579/806)	67.9% (2540/3742)

Coverage Summary for Package: com.partytime.service

Package	Class, %	Method, %	Branch, %	Line, %	Instruction, %
com.partytime.service	100% (19/19)	100% (62/62)	100% (30/30)	100% (284/284)	100% (1167/1167)

Class 	Class, %	Method, %	Branch, %	Line, %	Instruction, %
AccountDeletionService	100% (1/1)	100% (2/2)	100% (4/4)	100% (17/17)	100% (83/83)
AccountService	100% (2/2)	100% (9/9)	100% (4/4)	100% (42/42)	100% (173/173)
AccountServiceKt	100% (1/1)	100% (1/1)		100% (1/1)	100% (4/4)
AddressService	100% (1/1)	100% (4/4)		100% (17/17)	100% (56/56)
AuthService	100% (3/3)	100% (6/6)	100% (4/4)	100% (23/23)	100% (106/106)
AuthServiceKt	100% (1/1)	100% (1/1)		100% (1/1)	100% (4/4)
CryptService	100% (1/1)	100% (4/4)		100% (5/5)	100% (23/23)
JwtService	100% (3/3)	100% (10/10)	100% (10/10)	100% (32/32)	100% (152/152)
OrganizerService	100% (4/4)	100% (17/17)	100% (8/8)	100% (128/128)	100% (482/482)
OrganizerServiceKt	100% (1/1)	100% (1/1)		100% (1/1)	100% (4/4)
ParticipantService	100% (1/1)	100% (7/7)		100% (17/17)	100% (80/80)

Frontend

- Projektstruktur
- Services
- Komponenten
- Routen
- Validierung
- Tests
- Coverage

Projektstruktur

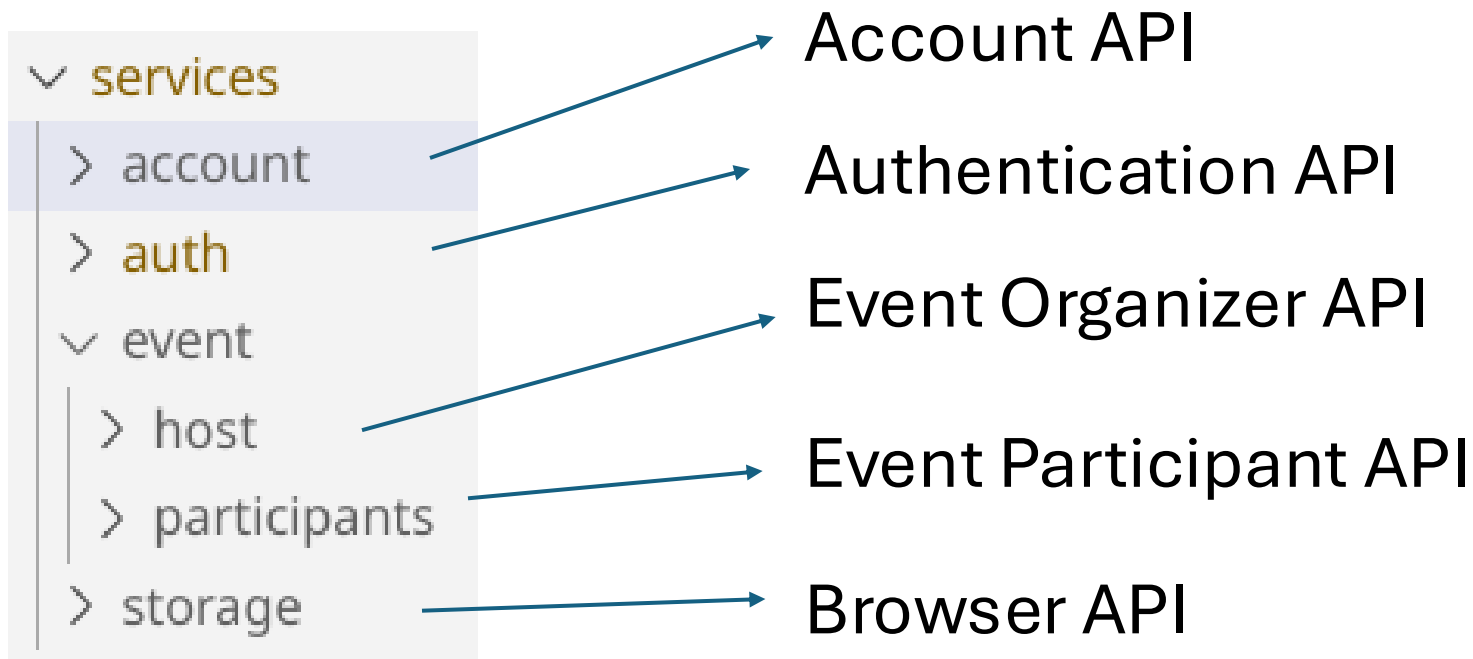
- "components" - Globale Komponenten
- "models" - Modelle (Interfaces)
- "pages" - Pages (Routen)
- "service" - Services (Verbindung zum Backend)
- "validators" - Validatoren

```

  ✓ src
    ✓ app
      > .storybook
      > components
      > models
      > pages
      > services
      > validators
      TS app.component.spec.ts
      TS app.component.ts
      TS app.config.ts
      TS app.routes.ts
      > assets
      > environments
      ★ favicon.ico
      <> index.html
      TS main.ts
      🍷 styles.scss
      TS test-setup.ts
```

Services

- Aufgeteilt nach Funktion
- Spiegeln die Swagger Dokumentation
- Werden getestet



Componenten

- Unterscheidung zwischen globalen und lokalen Komponenten
- Global:
 - Können von allen verwendet werden
 - Werden im Projekt exportiert
 - White Box im Projekt
- Lokal:
 - Können nur in einem begrenzten Scope verwendet werden
 - Werden nicht im Projekt exportiert
 - White Box im Scope, Black Box im Projekt

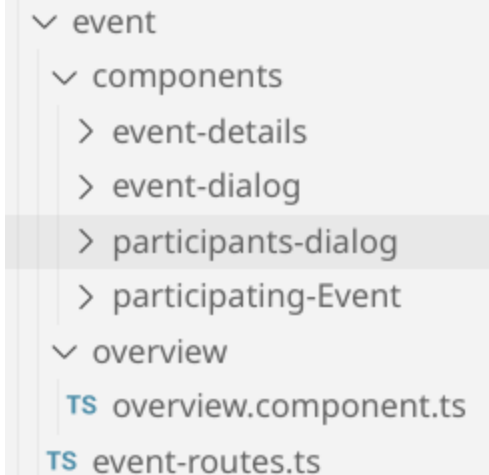
Routen

- Unterscheidung zwischen geschützten und ungeschützten Routen
- Lazy routen welche erst geladen werden wenn benötigt
- Children -> laden andere Routen
- Componenten -> tatsächliche Seite
- Catch All -> abfangen von ungültigen Routen

```
{  
  path: 'delete',  
  loadComponent: () =>  
    import('./delete/delete.component').then((c) => c.DeleteComponent),  
  title: 'Account löschen',  
  canActivate: [authGuard],  
},
```

Routen / Pages

- Kann eigene Teil-Routen haben
- Hat Zugriff auf eigene & globale Komponenten
- Kann Services injiziert bekommen
- Wird dem Nutzer angezeigt



```

└─ event
  └─ components
    ├── event-details
    ├── event-dialog
    └─ participants-dialog
  └─ participating-Event
  └─ overview
    ├── TS overview.component.ts
    └─ TS event-routes.ts

```

A screenshot of a file explorer showing a directory structure. The 'event' directory is expanded, showing subdirectories 'components' and 'overview'. The 'components' directory is further expanded, showing files 'event-details', 'event-dialog', and 'participants-dialog'. The 'overview' directory is also expanded, showing files 'TS overview.component.ts' and 'TS event-routes.ts'. The 'participants-dialog' file is highlighted.

Validierung FormGroup

- FormGroup mit Validierung basierend auf Design Entscheidung
- Beispiel Registrierung:

```
registerForm = new FormGroup({  
  name: new FormControl('', [  
    Validators.required,  
    Validators.minLength(5),  
    Validators.maxLength(20),  
  ]),  
  email: new FormControl('', [Validators.required, Validators.email]),  
  password: new FormControl('', [  
    Validators.required,  
    Validators.minLength(8),  
    Validators.maxLength(30),  
    CustomValidators.passwordStrength,  
  ]),  
});
```

Validierung Design-Entscheidung

E-Mail-Adresse Richtlinien

E-Mail Adressen müssen dem HTML Standard für Valide E-Mail Adressen entsprechen und vom *Angular* Framework als valide empfunden werden.

Begründung:

Das verwendete *Angular* Framework hat einen [email Validator](#) für diesen Einsatzzweck, der zur Validierung von E-Mail-Adressen in Formularen mit geringem Entwickleraufwand eingesetzt werden kann.

Passwort-Richtlinien für Benutzer

Passwörter für Benutzer müssen die folgenden Anforderungen erfüllen:

- Mindestens 8 Zeichen lang sein
- Maximal 30 Zeichen lang sein
- 4 Zeichenarten verwenden (je min. 1 Großbuchstaben, Kleinbuchstaben, Ziffer und Sonderzeichen)
- Die 1 benötigte Ziffer und das 1 benötigte Sonderzeichen dürfen nicht am Anfang oder Ende des Passworts stehen. Auch eine Folge von Sonderzeichen und Ziffern am Anfang und Ende erfüllen die Regel nicht.
- Als Sonderzeichen anerkannt werden: „!\"\$%&/()=?{}[]\ und Leerzeichen

Begründung:

Einhaltung der [Bundesamt für Sicherheit in der Informationstechnik \(BSI\): Sichere Passwörter erstellen](#) Empfehlung

Validierung Ergebnis

```
<mat-form-field appearance="fill" data-cy="password-field">
  <mat-label>Passwort</mat-label>
  <input
    type="password"
    matInput
    formControlName="password"
    data-cy="password-input"
  />
  <mat-error
    *ngIf="registerForm.get('password')?.errors?.['required']"
    data-cy="password-required-error"
  >
    Passwort ist erforderlich.
  </mat-error>
  <mat-error
    *ngIf="registerForm.get('password')?.errors?.['minlength']"
    data-cy="password-minlength-error"
  >
    Das Passwort muss mindestens 8 Zeichen lang sein.
  </mat-error>
  <mat-error
    *ngIf="registerForm.get('password')?.errors?.['maxlength']"
    data-cy="password-maxlength-error"
  >
    Das Passwort darf maximal 30 Zeichen lang sein.
  </mat-error>
  <mat-error
    *ngIf="registerForm.get('password')?.errors?.['uppercaseRequired']"
  >
    Mindestens ein Großbuchstabe ist erforderlich.
  </mat-error>
  <mat-error
    *ngIf="registerForm.get('password')?.errors?.['lowercaseRequired']"
  >
    Mindestens ein Kleinbuchstabe ist erforderlich.
  </mat-error>
  <mat-error
    *ngIf="registerForm.get('password')?.errors?.['numberRequired']"
  >
    Mindestens eine Zahl ist erforderlich.
  </mat-error>
  <mat-error
    *ngIf="registerForm.get('password')?.errors?.['specialCharRequired']"
  >
    Mindestens ein Sonderzeichen ist erforderlich.
  </mat-error>
</mat-form-field>
```

Account erstellen

Benutzername*

Benutzername ist erforderlich.

Email*

E-Mail ist erforderlich.

Passwort*

...



Das Passwort muss mindestens 8 Zeichen lang sein.

Mindestens ein Großbuchstabe ist erforderlich.

Mindestens eine Zahl ist erforderlich.

Mindestens ein Sonderzeichen ist erforderlich.

Registrieren

Tests

- Services werden anhand der API Dokumentation getestet
- Mock werden aufgebaut
- Services Instanziiert und Injiziert

```
beforeEach(() => {  
    TestBed.configureTestingModule({  
        imports: [HttpClientTestingModule],  
        providers: [EventHostService],  
    });  
    service = TestBed.inject(EventHostService);  
    httpTestingController = TestBed.inject(HttpTestingController);  
});  
afterEach(() => {  
    httpTestingController.verify();  
});
```

Tests II

- Methoden werden auf good & bad mit Assert getestet
- Jeder Pfad soll mindestens 1x durchlaufen werden

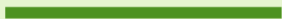
```
it('should get an event', () => { ...  
});
```

```
it('should handle errors when fetching a specific event', () => { ...  
});
```

```
2x  
getEvent(eventId: number): Observable<OrganizerEventDTO> {  
  return this.http  
    .get<OrganizerEventDTO>(  
      environment.api.endpoints.event.host.getEvent(eventId.toString())  
    )  
    .pipe(  
      catchError((error) => {  
1x        return throwError(() => error.error as ApiError);  
      })  
    );  
}
```

Coverage

- Statements -> 100%
- Branches -> 100%
- Functions -> 100%
- Lines -> 100%

app/models		100%	70/70	100%	2/2	100%	1/1	100%	70/70
app/models/dto		100%	4/4	100%	2/2	100%	1/1	100%	4/4
app/services/account		100%	20/20	100%	6/6	100%	7/7	100%	18/18
app/services/auth		100%	27/27	100%	4/4	100%	12/12	100%	23/23
app/services/auth/guard		100%	11/11	100%	1/1	100%	1/1	100%	10/10
app/services/auth/interceptor		100%	14/14	100%	4/4	100%	1/1	100%	13/13
app/services/event/host		100%	32/32	100%	0/0	100%	25/25	100%	22/22
app/services/event/participants		100%	19/19	100%	0/0	100%	12/12	100%	14/14
app/services/storage		100%	8/8	100%	0/0	100%	3/3	100%	6/6
app/validators		100%	18/18	100%	9/9	100%	2/2	100%	18/18
environments		100%	18/18	100%	0/0	100%	17/17	100%	18/18

End-to-End Tests

- E2E Tests mit Cypress und Live-Betrieb des Front-Ends & Back-Ends
- Tests bestätigen Abnahmekriterien der User-Stories
- Bestätigen Erfüllung der Anforderungen